Pre-training, Fine-tuning, and Prompting Strategies for Bug Fixing

Yibarek Tadesse

April 2026

Course: GenAI for Software Development (CSCI 455/555)
Instructor: Dr. Antonio Mastropaolo

Abstract

This report investigates the impact of pre-training and prompting strategies on code bug-fixing tasks. We evaluate four distinct configurations: a T5-small model pre-trained on a Java corpus and fine-tuned for bug fixing (Pipeline A), a T5-small model fine-tuned without pre-training (Pipeline B), a Retrieval-Augmented Generation (RAG) 3-shot prompting strategy using Qwen 1.5B (Pipeline C - Few Shot), and a zero-shot Qwen 1.5B baseline (Pipeline C - Zero Shot). The pre-trained T5 model performs similarly to the model trained from scratch, yielding CodeBLEU scores of ∼56.9. Surprisingly, zero-shot prompting vastly outperforms few-shot RAG (19.29 vs 7.10 CodeBLEU), indicating that retrieved examples introduced significant noise or context misalignment for the generator model.

1 Dataset Construction

To train and evaluate the models, two primary datasets were leveraged. For the pre-training phase, 50,000 Java methods were extracted from the CodeSearchNet dataset. The downstream task of bug fixing utilized the medium split of the CodeXGLUE code refinement dataset, consisting of pairs of buggy and corresponding fixed Java methods. The splits contained 52,364 pairs for training, 6,546 pairs for validation, and 6,545 pairs for testing.

2 Preprocessing & Tokenization

A custom SentencePiece tokenizer was trained directly on the CodeSearchNet pre-training corpus using the Unigram subword algorithm. The vocabulary size was capped at 16,384 tokens. Additionally, T5 special tokens (such as `<pad>`, `</s>`, `<unk>`, and sentinel tokens `<extra_id_0>` through `<extra_id_99>`) were added to support the span corruption objective.

During pre-training, a span corruption objective was employed with a corruption rate of 15% and a mean span length of 3 tokens. Randomly selected token spans were replaced with sentinel tokens.

3 Model Design and Training

3.1 Pipeline A & B (T5-Small)

In Pipeline A, a T5-small architecture (∼60M parameters) was pre-trained on the CodeSearchNet corpus using the span corruption objective for 3 epochs, and subsequently fine-tuned on the CodeXGLUE bug-fixing dataset for 3 epochs. Pipeline B served as a baseline, initializing an identical T5-small model from scratch and fine-tuning it directly on the bug-fixing dataset, bypassing pre-training.

3.2 Pipeline C (RAG-based Generation)

Pipeline C explored prompting a larger, pre-trained instruct model (Qwen2.5-Coder-1.5B-Instruct). A RAG system was constructed using CodeBERT embeddings and FAISS for fast nearest-neighbor search. The following snippet demonstrates the few-shot prompt formulation logic used to provide the generator with contextually relevant bug-fix pairs:

```python
def build_few_shot_prompt(self, buggy_code: str, retrieved_examples:
    List[Dict], num_shots: int = 3) -> str:
    prompt = "Fix the following Java method bug:\n\n"
    if retrieved_examples:
        prompt += "Examples of similar bug fixes:\n"
        for i, example in enumerate(retrieved_examples[:num_shots], 1):
            prompt += f"\nExample {i}:\n"
            prompt += f"Buggy:\n{example['buggy'][:200]}...\n"
            prompt += f"Fixed:\n{example['fixed'][:200]}...\n"

    prompt += f"\n\nNow fix this code:\nBuggy:\n{buggy_code}\n\nFixed:"
    return prompt
```

Listing 1: Few-Shot Prompt Construction

4 Evaluation and Results

Two primary metrics were used: Exact Match and CodeBLEU. Exact Match measures the strict percentage of predictions that perfectly replicate the target fixed code. CodeBLEU is a code-aware metric that combines standard N-gram matching with syntactic AST similarities and semantic dataflow matching.

4.1 Comparative Analysis

The evaluation yielded the following results across the tested configurations:

Pipeline Configuration	Exact Match (%)	CodeBLEU
A: Pre-trained + Fine-tuned	0.00	56.90
B: Fine-tuned (No Pre-training)	0.00	56.86
C: RAG (With Retrieval, 3-shot)	0.00	7.10
C: RAG (Zero-shot)	0.00	19.29

Table 1: Code Fixing Metric Evaluation across Pipelines

4.2 Important Findings

The results highlight several important observations regarding model training dynamics for bug fixing:

- **Contextualizing the 0% Exact Match:** While an Exact Match score of 0.00% across all pipelines highlights the extreme rigidity of the metric. Exact Match requires verbatim, token-for-token replication of the target reference fix. Given the subjective nature of programming, a bug can often be resolved in multiple functionally equivalent ways (e.g., alternative variable names, different loop structures, varying spacing). The strong CodeBLEU scores achieved by the fine-tuned models ($\sim$56.9) confirm that they are successfully learning the underlying structures of valid Java code, even if their proposed fixes do not perfectly mirror the specific human-authored ground truth.

- **Marginal Impact of Pre-training:** Pipelines A and B reached strong and nearly identical CodeBLEU scores (56.90 vs 56.86). This suggests that for this specific task and scale, the span corruption pre-training phase did not yield a statistically significant advantage over fine-tuning from scratch.

- **Fine-tuning Superiority over Prompting:** The 60M-parameter fine-tuned models vastly outperformed the 1.5B-parameter Qwen generator on the CodeBLEU metric.

- **Negative Impact of RAG Retrieval:** Surprisingly, the zero-shot baseline significantly outperformed the few-shot retrieved prompting (19.29 vs 7.10 CodeBLEU). This suggests that the retrieved CodeXGLUE examples likely introduced noise or misaligned context for the Qwen 1.5B model, diluting its zero-shot bug-fixing capabilities.

5 Reproducibility & Repository

To ensure reproducibility, all random seeds across numpy, Python's random module, and PyTorch were fixed to 42. Code implementations are thoroughly documented within the project workspace. For more information regarding the data implementations of the source code, visit the repo attached to this link: `https://github.com/Yibarek1/Strats-for-Bug-Fixing`